



React Native Assignment

Role: React Native Mobile Developer

Time Expectation: 6–8 hours (do not exceed)

Submission Window: 48–72 hours

AI Tools: Allowed (with disclosure)

Objective

Build a small mobile app that demonstrates:

- Clean React Native structure
- Correct API integration and state management
- Sensible navigation and screen composition
- Reliable handling of loading, error, and empty states
- Ability to explain decisions and trade-offs

This is not a UI showcase.

This is about correctness and predictability.

Problem Statement

Build a **Project Tracker Mobile App** that consumes a backend API and allows users to view and update project status.

You may:

- Use the backend you built earlier, **or**
 - Use a mock API / local JSON server (document which you chose)
-

Core Entity: Project

Each project has:

- `id`
- `name`
- `clientName`
- `status` (`active` , `on_hold` , `completed`)
- `startDate`
- `endDate` (optional)

You may add **one additional field** if justified and documented.

Functional Requirements

1. Project List Screen

- Display a list of projects
 - Show: name, client, status
 - Support pull-to-refresh
 - Handle long names without breaking layout
-

2. Filters & Search

- Filter by status
 - Search by project or client name
 - Filters must work together
 - Original dataset must remain intact
-

3. Project Detail Screen

- Navigate on project tap
- Show all project details

- Allow updating project status
 - Reflect changes immediately in UI
-

4. States & Edge Cases

Must handle:

- Loading states
- Empty list
- No results after filtering
- API failure (network error / invalid response)

No crashes. No frozen UI.

Technical Constraints

- React Native (CLI or Expo)
 - Functional components only
 - Hooks for state management
 - React Navigation (or equivalent)
 - No heavy UI kits that hide logic
 - Clean separation of screens, components, and services
-

Code Quality Expectations

We will review:

- Folder structure
- Component responsibility boundaries
- Naming clarity
- API/service abstraction
- Readability and maintainability

Over-engineering is a negative signal.

AI Usage Policy (Mandatory Disclosure)

You may use AI tools.

Include a section in `README.md` answering:

- Which tools were used
- For which parts
- What you changed or rejected
- What you fully understand vs partially

In the next round, you must be able to explain the entire codebase.

Submission Requirements

1. GitHub repository
 2. `README.md` with:
 - Setup instructions
 - API/mocking approach
 - Assumptions and trade-offs
 3. Optional (bonus):
 - APK / TestFlight build
 - Short screen recording
 - Basic tests
-

Evaluation Criteria

Area	Weight
Data fetching & state handling	High
Navigation & screen logic	High

Area	Weight
Edge case handling	High
Code structure	High
Reasoning & explanation	Medium
Visual polish	Low

Disqualifiers

- Hardcoded UI without real data flow
 - Unhandled loading or error states
 - Code you cannot explain
 - Excessive libraries without justification
-

What This Assignment Is Testing

- Can you build a stable mobile app, not just screens
- Can you reason about async data and UI state
- Can you ship something explainable and reliable